```
request, "/spaces/" + spaceId + "/messages", "rw"); #A
var messagesReadOnlyUri = capabilityController.createUri( #A
request, "/spaces/" + spaceId + "/messages", "r"); #A
response.status(201);
response.header("Location", uri.toASCIIString());
return new JSONObject()
.put("name", spaceName)
.put("uri", uri)
.put("messages-rwd", messagesUri) #B
.put("messages-rw", messagesReadWriteUri) #B
.put("messages-r", messagesReadOnlyUri); #B
```

#A Create additional capability URIs with restricted permissions.
#B Return the additional capabilities.

To complete the conversion of the API to capability-based security you need to go through the other API actions and convert each to return appropriate capability URIs. This is largely a straightforward task, so we won't cover it here. One aspect to be aware of is that you should ensure that the capabilities you return do not grant more permissions than the capability that was used to access a resource. For example, if the capability used to list messages in a space granted only read permissions then the links to individual messages within a space should also be read-only. You can enforce this by always basing the permissions for a new link on the permissions set for the current request, as shown in listing 9.7 for the `findMessages` method. Rather than providing read and delete permissions for all messages, you instead use the permissions from the existing request. This ensures that users in possession of a moderator capability will see links that allow both reading and deleting messages, while